Randy Zhu

Vancouver, BC | randy@randyzhu.com | github.com/RandoNandoz | linkedin.com/in/rzhu08 | Canadian Citizen · Open to Relocation

Education

University of British Columbia April 2028
Bachelor of Science, Computer Science, Option in Software Engineering *GPA: 3.88/4*
Teaching Assistant for Introduction to Computer Systems for 400+ students · Undergraduate TA Award

Experience

Firmware Engineer Intern May 2026 – Present
Netgear *Vancouver, BC*

- Cut firmware RAM by 400MB and eMMC footprint by 600MB, enough to eliminate a planned memory upgrade on Netgear's flagship Wi-Fi 7 router; **projected 2 million dollar hardware upgrade avoided**
- Eliminated the need to flash to hardware test cycle for most UI changes with a host-side build target that runs the firmware UI on a dev machine, mocking the backend and replacing the on-device fbdev, libinput and evdev backend with SDL2
- Improved crash logs by replacing GCC with clang in the debug UI build, enabling support for AddressSanitizer and other sanitizers for rich memory error and undefined behavior error information
- Fixed a 2-year-old display tearing bug on device display by tracing it to an unconnected tearing-effect line, then exposing the GPIO in the device tree and gated frame writes on interrupt in the Linux kernel driver

Research Intern May 2025 – September 2025
University of British Columbia *Vancouver, BC*

- Achieved a median speedup of 4.97x on large, open-source codebases with long-running integration tests by building a unit test generation tool targeting pytest using Python using dynamic execution tracing, AST transformations, and serialization
- Identified correct assertion generation, not execution capture, as the key barrier to automating carving on real test suites
- Integrated an LLM as a dependency classifier to auto-generate mocks for creating correct and deterministic unit tests

Software Developer Intern September 2024 – April 2025
Teck Resources *Vancouver, BC*

- Eliminated ~100 hours of manual data entry by building a Dataverse-API ingest tool in C#
- Transformed a malformed SQL Server database with multivalued records stored as CSV strings in single columns into actionable business and safety insights, using T-SQL to parse the columns into separate columns and normalized tables
- Introduced the team to Playwright + NUnit to automate end-to-end testing, catching 87% of bugs before manual QA
- Deployed a React.js-based calendar component across all Power Apps teams, implemented in TypeScript

Projects

macOS iSCSI Driver | *C++, DriverKit, IOKit, Swift* November 2025 – Present

- Building an open-source macOS iSCSI initiator that mounts remote storage as a local disk, providing a free, auditable alternative to $249/license proprietary tools
- Bridging the Swift userspace networking client to the driver extension over a lock-free shared-memory ring buffer, saturating 2 10Gb SFP+ NICs, working around DriverKit's lack of network access
- Exposing the remote target as a local block device via the macOS IOUserSCSIParallelInterfaceController interface

Racket Compiler | *x86_64 Assembly, LLVM IR, gdb, Racket* January 2026 – May 2026

- Cut compile times 180x (3h to 1m) by eliminating a pathological typed/untyped bottleneck in Typed Racket
- Replaced x86-only codegen with an LLVM IR backend, enabling compilation to any LLVM-supported platform
- Implemented closure conversion, tagged value representation, and tail call optimization via jump conversion

Sasquatch | *Swift, ARKit, LiDAR, Detectron2, Open3D, Python, FastAPI, PostgreSQL, GCP* March 2026 (24h Hackathon)

- Deployed an ML pipeline combining Detectron2 instance segmentation and LiDAR point clouds to detect climbing wall holds
- Implemented PCA-based wall orientation detection to project 3D point clouds to 2D for inference
- **Reduced false positive rate from 90% to 1%** by adding a Gemini validation pass to filter non-hold wall objects

Skills

Languages: C, C++, Python, Rust, TypeScript, Swift, Java, C#, SQL, Racket
Technologies: LVGL, React, Embedded Linux, OpenWRT, LLVM IR, DriverKit, IOKit
Cloud/Infra: AWS (EC2, ECS), GCP (Cloud Run, Cloud SQL, Gemini), Cloudflare (DNS, Pages), Docker, Linux
Tools/Testing: Git, GitHub Actions, CMake, GDB, Ghidra, pytest, Playwright, NUnit, JUnit